Quiz Maker

Python + SQLite capstone project with file import, role-based access, randomized quizzes, and full admin CRUD.

Project summary

- Creates quizzes from a SQLite question bank
- Imports questions from CSV into the database
- Registers users and stores password hashes with Argon2
- Uses getpass so passwords do not display in the terminal
- Saves quiz results for later review
- Lets admins manage users and questions through the menu

10-minute presentation plan

1) problem and goals
2) design and rubric coverage
3) setup + CSV import
4) student flow
5) admin flow
6) security + conclusions

End-to-end workflow

1. Setup DB — Creates Users, Questions, and QuizResults tables plus the starting admin account.

2. Import CSV — Reads questions.csv and inserts 30 question records into SQLite.

What the live walkthrough shows

```
$ python db_setup.py
Users / Questions / QuizResults tables created
$ python import_questions.py
30 questions loaded from CSV
$ python quiz_app.py
admin login -> register user -> take quiz -> view scores
promote user to admin -> add/delete question -> delete user
```

How the application works

The project combines three data entities, Python classes, CSV file input, and an interactive menu.

Student flow

- Register a new account or log in.
- Take a randomized 10-question quiz.
- Receive immediate right/wrong feedback.
- Quiz score is stored in QuizResults.
- Use View My Scores to review prior attempts.

Admin flow

- Log in as admin from the same menu system.
- View all questions and users.
- Add, edit, and delete questions.
- Promote standard users to admin.
- Reset passwords and delete users when needed.

Security + quality

- Passwords are hidden with getpass.
- Passwords are stored as Argon2 hashes instead of plain text.
- Input validation rejects invalid quiz answers.
- Cancel prompts reduce accidental admin changes.
- The database is not static because menu actions update data live.

Data model used by the app

Users

user_id, username, password_hash, is_admin

Questions

question_id, prompt, choice_a-d, correct_answer

QuizResults

result_id, user_id, score, total_questions, date_taken

These 3 tables satisfy the rubric requirement for at least 3 entities, and the transcript shows 30 imported questions plus user/result rows during the demo.

Rubric alignment at a glance

This version of the presentation explicitly shows the functionality demonstrated in the terminal walkthrough.

Documentation	Inline comments explain the workflow, and the presentation includes approach, design, ethics, and critical thinking.
SQL + CRUD	Users and questions support create, retrieve, update, and delete actions from the menu.
File I/O	questions.csv is opened, parsed, confirmed, and inserted into SQLite.
Error handling	The app handles invalid IDs, invalid quiz answer choices, missing users, and cancel actions.
Object oriented	Separate classes manage the database tables while QuizApp handles user interaction.
Interactive menu	Standard users see quiz options, while admins also see management options.
Presentation quality	Slides follow a clear story: setup, student flow, admin flow, security, and proof of results.

Suggested speaking point: "Our live walkthrough now proves registration, login, role changes, question CRUD, score storage, and user deletion."

Demo 1 - environment setup and CSV import

The first part of the walkthrough proves that the database is created and populated dynamically from a CSV file.

Database setup and CSV import

```
$ python db_setup.py
Users table created successfully.
Questions table created successfully.
QuizResults table created successfully.
Database setup complete.
User 'admin' added successfully.
Admin user created.

$ python import_questions.py
Successfully read 30 questions from questions.csv
Number of questions to save: 30
Continue? (y/n) y
Import complete!
```

What the terminal output proves

- A virtual environment is created and requirements are installed.
- db_setup.py creates the Users, Questions, and QuizResults tables.
- An initial admin user is inserted into the Users table.
- import_questions.py reads questions.csv and saves 30 question rows into SQLite.
- This satisfies both the database and file-I/O parts of the rubric.

Suggested timing: about 1 minute

Demo 2 - authentication, registration, and student quiz flow

This portion of the run shows secure login, role-based menus, randomized quiz questions, and saved scores.

Student actions shown

- Register a new user (jorton4).
- Login succeeds with password hidden in the terminal.
- Standard user menu shows only quiz, scores, and logout.
- Take a 10-question randomized quiz.
- Score is saved to QuizResults and later displayed.

Security upgrade shown in code

- getpass hides password entry.
- Argon2 hashes passwords before storage.
- Login verifies the stored hash instead of comparing plain text.

Transcript highlights

```
=== Register ===
User 'jorton4' added successfully.
=== Quiz Complete! ===
Your score: 0/10
Date: 2026-04-16 ... | Score: 0/10
```

Demo 3 - admin user management

Admins can manage accounts without touching the database manually. The transcript shows promotion, password reset, and deletion.

```
Select an option:
=== Add a Question ===
(T
...
=== Add a Question ===
(Type 'cancel' at any time to go back)

Enter question: Enter choice A: Enter choice B: Enter choice C: Enter choice D: Enter correct answer (A/B/C/D): Question a

=== Main Menu === (Logged in as: admin)
1. Take a Quiz
2. View My Scores
3. Logout
--- Admin ---
4. Add a Question
5. Edit a Question
6. Delete a Question
7. View All Questions
8. View All Users
9. Edit a User
10. Delete a User
Select an option:
=== Edit a Question ===
(Type 'cancel' at any time to go back)


=== All Questions ===
[1] What planet is closest to the sun?
[2] How many sides does a hexagon have?
[3] What is the capital of Japan?
[4] Which ocean is the largest?
[5] How many continents are on Earth?
[6] What is the chemical symbol for water?
[7] Who wrote Romeo and Juliet?
[8] How many hours are in a day?
[9] What color do you get when you mix red and white?
[10] What is the largest mammal in the world?
[11] How many days are in a leap year?
[12] What is the fastest land animal?
[13] Which planet is known as the Red Planet?
[14] How many bones are in the adult human body?
[15] What is the capital of Australia?
[16] Who painted the Mona Lisa?
[17] How many players are o
...
=== All Users ===
```

User management features demonstrated

- View all users with role labels.
- Edit a user to change username, password, and admin status.
- Promote a standard user to admin by setting is_admin = y.
- Delete a user account after confirmation.

Promotion example from the transcript

```
Enter user ID to edit: 2
Enter new username [jorton4]:
Leave password blank to keep the current password.
Is this user an admin? (y/n) [n]: y
User updated successfully.
```

Proof that the role changed

```
=== All Users ===
[1] admin - Admin
[2] jorton4 - Admin

Logging in as jorton4 now shows the admin menu.
```

Demo 4 - admin question CRUD and data changes

The app also demonstrates question management directly from the interactive menu.

Question CRUD shown live

- View all questions already loaded from the CSV file.
- Add a new test question through the admin menu.
- View the new question in the question list.
- Delete the question with confirmation.
- View the list again to prove the deletion worked.

Add question snippet

```
=== Add a Question ===
Enter question: Here is my test question?
Enter choice A: yes
Enter choice B: no
Enter choice C: maybe
Enter choice D: idk
Enter correct answer (A/B/C/D): a
Question added successfully.
```

Delete question snippet

```
=== Delete a Question ===
Enter question ID to delete: 31
Are you sure you want to delete question
31? (y/n): y
Question ID 31 deleted successfully.
```

Proof of the data changing

```
Before delete:
[31] Here is my test question? (Answer
A)

After delete:
Question list ends at [30] What year did
World War II end?
```

This proves the app is interacting with SQLite live rather than using static data.

Error handling, ethics, and critical thinking

The code goes beyond a happy-path demo by validating input and reducing risky actions.

Error handling

- Rejects invalid quiz answers until the user enters A, B, C, or D.
- Catches non-numeric IDs during edit and delete operations.
- Shows “User ID not found” or “Question ID not found” when records do not exist.
- Uses cancel prompts to back out of admin actions safely.

Ethics / critical thinking

- Using Argon2 and getpass protects credentials better than storing plain text passwords.
- Role-based access keeps standard users from seeing admin controls.
- Delete actions ask for confirmation to avoid accidental data loss.
- Stored quiz results provide an honest record of student performance.

OOP and collaboration value

- Separate classes organize DB logic and application logic.
- This structure lets teammates work on setup, import, app flow, and admin logic separately.
- Inline comments explain purpose and improve maintainability.

Next improvements

- Add better UI formatting for long terminal output.
- Improve password reset UX with confirmation.
- Add question categories or difficulty levels.
- Create a future web front end on top of the same database design.

Suggested closing demo script

This slide helps keep the presentation under 10 minutes while covering all required functionality.

0:00-1:00	Introduce the problem: instructors need reusable quizzes, student accounts, and saved results.
1:00-2:00	Explain the 3-table SQLite design and how CSV import populates the database.
2:00-3:30	Show db_setup.py and import_questions.py running successfully.
3:30-5:30	Register a student, take a randomized quiz, and view saved scores.
5:30-8:00	Log in as admin, show users, promote a user to admin, and discuss password hashing.
8:00-9:15	Add and delete a question to prove CRUD against SQLite.
9:15-10:00	Close with rubric alignment, ethics, and future improvements.

Final takeaway: the project now demonstrates setup, import, secure authentication, student quiz-taking, score persistence, admin promotion, admin CRUD, and deletion end to end.